

Unsupervised Network Intrusion Anomaly Detection Using Autoencoder Neural Networks: A Comparative Study

Ann Hoang, Hongyan Chang, Manisurya Udhisi, Drashtiben Vaghasiya, and Ziqi Guo

Department of Electrical, Computer and Systems Engineering

McMaster University, Hamilton, ON, Canada

Abstract—Modern Network Intrusion Detection Systems (NIDS) face critical operational challenges driven by the increasing frequency, sophistication, and volume of cyber threats. Supervised machine learning algorithms often struggle in real world deployment due to severe class imbalance and an inability to flag uncatalogued zero-day exploits. Unsupervised anomaly detection via autoencoder neural networks mitigates these issues by learning a compact latent representation of legitimate network traffic and identifying malicious intrusions through elevated reconstruction errors. This paper provides a comprehensive empirical evaluation of three autoencoder architectures: a Basic Autoencoder baseline, a Sparse Autoencoder regularized via Kullback-Leibler (KL) divergence, and an expanded Deep Autoencoder. Using the preprocessed KDD Cup 1999 dataset [1], models were trained exclusively on normal traffic sessions and evaluated against 57,751 unseen anomaly records across four attack vectors (DoS, Probe, R2L, and U2R). Utilizing calibrated percentile thresholds on normal traffic, both Basic and Sparse Autoencoders achieved exceptional operational performance with F1-scores up to 0.9876. Notably, enforcing latent activation sparsity improved sensitivity to low-volume, stealthy Remote-to-Local (R2L) intrusions (recall of 0.952). Conversely, expanding network depth without structural regularization degraded detection performance on subtle attacks (R2L recall dropping to 0.510), revealing critical design insights regarding latent capacity, optimization gradients, and regularization in deep autoencoders.

Index Terms—Network Intrusion Detection, Unsupervised Anomaly Detection, Deep Autoencoders, Sparse Autoencoders, Kullback-Leibler Divergence, Reconstruction Loss.

I. INTRODUCTION

Cybersecurity infrastructures depend heavily on Automated Intrusion Detection Systems (NIDS) to protect enterprise networks from unauthorized access, service disruption, and data exfiltration. NIDS frameworks fall into two primary paradigms: signature-based detection and anomaly-based detection. Signature-based systems monitor packet headers and payload contents against static databases of known attack fingerprints. While highly precise for recognized threats, signature matching fails completely when encountering modified attack strains or zero-day vulnerabilities.

In contrast, anomaly-based NIDS establishes a statistical or machine-learned model of benign network behavior. Any session or connection displaying significant deviation from this baseline is flagged as an anomaly. Unsupervised deep learning—specifically using autoencoder neural networks—has emerged as a leading methodology for anomaly detection

[2], [3]. Autoencoders operate by mapping high-dimensional input vectors into a compressed lower-dimensional bottleneck representation before attempting to reconstruct the original input. When trained exclusively on legitimate network traffic, the network captures the underlying feature correlations of normal operational patterns. Consequently, when presented with malicious network traffic during inference, the autoencoder struggles to accurately reconstruct the unfamiliar patterns, resulting in significantly higher reconstruction errors.

Despite their success, autoencoder-based anomaly detectors face several open research questions regarding network depth, activation sparsity, and threshold sensitivity [2]. This paper conducts a rigorous comparative analysis across three distinct autoencoder variants:

- 1) **Basic Autoencoder:** A compact, symmetric 2-hidden-layer network serving as our baseline.
- 2) **Sparse Autoencoder:** An architecture enforcing a Kullback-Leibler (KL) divergence penalty on latent activations to promote feature specialization [4].
- 3) **Deep Autoencoder:** A 4-hidden-layer feedforward network built to examine whether hierarchical abstraction improves separation between normal and malicious traffic.

The remainder of this paper is organized as follows: Section II reviews related literature and theoretical foundations. Section III details the data cleaning, feature engineering, and splitting methodology applied to the KDD Cup 1999 dataset. Section IV presents the mathematical formulations and architectural configurations for all three models. Section V outlines the experimental results, threshold calibration dynamics, and per-attack-category performance breakdowns. Section VI discusses key analytical findings, and Section VII concludes the paper with directions for future work.

II. THEORETICAL BACKGROUND & RELATED WORK

A. Autoencoder Neural Networks

An autoencoder is a feedforward neural network comprising an encoder f_θ and a decoder g_ϕ . Given an input space $\mathcal{X} \in \mathbb{R}^d$, the encoder transforms an input vector x into a latent code $z \in \mathbb{R}^k$ (where $k < d$ enforces a spatial bottleneck):

$$z = f_\theta(x) = \sigma(W_e x + b_e) \quad (1)$$

where W_e and b_e represent weight matrices and bias vectors, and $\sigma(\cdot)$ denotes an element-wise non-linear activation function.

The decoder g_ϕ maps the latent representation z back to the original feature dimension to produce a reconstruction $\hat{x} \in \mathbb{R}^d$:

$$\hat{x} = g_\phi(z) = \psi(W_d z + b_d) \quad (2)$$

where W_d , b_d , and $\psi(\cdot)$ denote decoder parameters and activation functions. Parameters $\Theta = \{\theta, \phi\}$ are optimized by minimizing a reconstruction loss function over N normal training samples:

$$\mathcal{L}_{\text{MSE}}(x, \hat{x}) = \frac{1}{d} \sum_{j=1}^d (x_j - \hat{x}_j)^2 \quad (3)$$

B. Sparsity Regularization in Autoencoders

Standard autoencoders risk copying input features directly if the latent dimension k is overly large or if the network has high capacity. Sparse autoencoders constrain the network by forcing most hidden neurons to remain inactive for any given input [4].

Let $a_j(x)$ denote the activation of latent neuron j for input sample x . The average activation of neuron j over a mini-batch of size M is defined as:

$$\hat{\rho}_j = \frac{1}{M} \sum_{i=1}^M a_j(x^{(i)}) \quad (4)$$

To enforce sparsity, $\hat{\rho}_j$ is constrained to match a small target sparsity value ρ (typically $\rho \leq 0.10$). The penalty is quantified using the Kullback-Leibler (KL) divergence between two Bernoulli random variables:

$$\text{KL}(\rho \parallel \hat{\rho}_j) = \rho \log \frac{\rho}{\hat{\rho}_j} + (1 - \rho) \log \frac{1 - \rho}{1 - \hat{\rho}_j} \quad (5)$$

The total loss minimized during training balances reconstruction accuracy with latent sparsity weighted by hyperparameter β :

$$\mathcal{L}_{\text{Total}} = \mathcal{L}_{\text{MSE}}(x, \hat{x}) + \beta \sum_{j=1}^k \text{KL}(\rho \parallel \hat{\rho}_j) \quad (6)$$

C. Related Work in NIDS Anomaly Detection

Machine learning for intrusion detection has evolved from basic statistical clustering to deep generative and reconstructive modeling [5]. Early approaches utilized Principal Component Analysis (PCA) and One-Class Support Vector Machines (OC-SVMs). However, linear PCA fails to capture non-linear relationships, and OC-SVMs exhibit poor scaling quadratic to sample sizes.

Standard deep autoencoders have demonstrated strong results across benchmark datasets such as NSL-KDD and UNSW-NB15 [6]. Research by Shone et al. demonstrated that non-symmetric deep autoencoders achieve solid classification efficiency when coupled with shallow classifiers. However, existing literature frequently overlooks threshold calibration dynamics and per-attack-category recall sensitivities—specifically for rare, stealthy attack vectors like Remote-to-Local (R2L) and User-to-Root (U2R). This study addresses these gaps through systematic threshold calibration and ablation sweeps.

III. DATASET ARCHITECTURE & PREPROCESSING

A. The KDD Cup 1999 Benchmark

The experiments utilize the KDD Cup 1999 network dataset [1], consisting of simulated TCP/IP connection records. Each record represents a network session described by 41 input attributes (38 numeric variables and 3 categorical attributes) alongside a label designating the connection as either `normal` or a specific attack type. Malicious traffic falls into four main families:

- **Denial of Service (DoS):** Attacks attempting to disrupt system availability by flooding bandwidth or memory resources (e.g., `neptune`, `smurf`, `back`).
- **Surveillance / Probe:** Port scanning and network reconnaissance activities aimed at mapping active hosts and services (e.g., `satan`, `ipsweep`).
- **Remote to Local (R2L):** Unauthorized access attempts from a remote machine to compromise local account credentials (e.g., `guess_passwd`, `warezmaster`).
- **User to Root (U2R):** Local non-privileged access escalation attempts targeting superuser root privileges (e.g., `buffer_overflow`, `loadmodule`).

B. Data Cleaning and Deduplication Workflow

The raw KDD Cup training file contains 494,021 connection records. Data quality inspection confirmed zero missing or non-finite values. However, severe record redundancy was identified: 348,435 exact duplicate rows were detected across the feature space. Retaining duplicates causes artificial data skew, distorts cross-validation splits, and leads to optimistic evaluation metrics.

Additionally, cross-attribute conflict checks revealed 4 ambiguous records featuring identical 41-dimensional input values but contradictory labels (labeled simultaneously as normal traffic and attack). All duplicate and conflicting records were purged, leaving a clean core set of 145,582 unique records.

C. Role-Based Dataset Partitioning

To uphold strict unsupervised learning principles, normal traffic records were segregated from attack records:

- **Normal Pool:** 87,831 cleaned normal connection records.
- **Anomaly Reference Pool:** 57,751 cleaned attack records reserved strictly for final testing.

The normal pool was partitioned using a fixed random seed (42) into three disjoint subsets: 70% for training, 15% for validation (early stopping/hyperparameter selection), and 15% for threshold calibration. Table I details the final data partitioning.

D. Feature Engineering and Scaling

To eliminate feature-scale imbalance without introducing data leakage, a Scikit-learn [7] `ColumnTransformer` was fitted ****exclusively on the normal training split****. All parameters were subsequently applied to transform validation, calibration, and test arrays.

The feature transformation pipeline proceeded as follows:

TABLE I
PARTITIONING AND OPERATIONAL ROLES OF PREPROCESSED DATA

Data Split	Record Count	Operational Pipeline Role
Normal Training	61,481	Autoencoder Weight Fitting
Normal Validation	13,175	Early Stopping & Model Selection
Normal Calibration	13,175	Threshold Determination (τ)
Anomaly Reference	57,751	Evaluation Across Attack Classes

- 1) **Logarithmic Transformation** ($\log(1 + x)$): Nine highly right-skewed non-negative continuous features (e.g., duration, src_bytes, dst_bytes, count, srv_count) exhibited extreme values spanning multiple orders of magnitude. Applying $\log(1+x)$ compressed wide variances while preserving zero values.
- 2) **Min-Max Normalization**: All continuous numerical features were scaled to the interval $[0, 1]$:

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (7)$$

- 3) **Categorical One-Hot Encoding**: Categorical attributes (protocol_type, service, flag) were transformed into one-hot binary vectors with handle_unknown='ignore' enabled to safely process unseen categories.

After feature expansion, each network connection was transformed into a 74-dimensional single-precision float32 vector bounded within $[0, 1]$. Preprocessed feature arrays, preprocessor objects, and JSON metadata were serialized to the project artifacts/ directory.

IV. MODEL ARCHITECTURES & PIPELINE DESIGN

All three models share a modular design pipeline. Models are trained on X_train_normal, validated against X_validation_normal, calibrated on X_calibration_normal, and evaluated against X_anomaly_reference.

A. Basic Autoencoder Configuration

The basic autoencoder features a symmetric structure written in PyTorch [8]. The network contracts the 74-dimensional input into a 16-dimensional bottleneck before restoring the 74-dimensional output:

$$74 \xrightarrow{\text{Dense}} 64 \xrightarrow{\text{Dense}} 32 \xrightarrow{\text{Dense}} 16 \xrightarrow{\text{Dense}} 32 \xrightarrow{\text{Dense}} 64 \xrightarrow{\text{Dense}} 74 \quad (8)$$

All hidden layers employ Rectified Linear Unit (ReLU) activations ($\text{ReLU}(z) = \max(0, z)$). The final output layer uses a Sigmoid function ($\sigma(z) = (1 + e^{-z})^{-1}$) to match the $[0, 1]$ scaling of preprocessed inputs. Optimization is performed using Adam [9] (learning rate = 10^{-3} , weight decay = 10^{-5}) with mini-batch sizes of 512. Early stopping monitors validation loss with a patience of 8 epochs.

B. Sparse Autoencoder Configuration

The sparse autoencoder shares the $74 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 32 \rightarrow 64 \rightarrow 74$ layer dimension with the baseline. However, every encoder layer utilizes Sigmoid activations to restrict hidden outputs within $(0, 1)$, enabling batch probability estimation for KL divergence [4].

The training objective adds the KL penalty across latent activation probabilities to the MSE reconstruction loss. Hyperparameters $\beta = 10^{-3}$ and target sparsity $\rho = 0.10$ were selected following hyperparameter grid sweeps.

C. Deep Autoencoder Configuration

The deep autoencoder expands network depth to four hidden layers per side, yielding an eleven-layer feedforward network built in TensorFlow/Keras [10]:

$$74 \rightarrow 64 \rightarrow 48 \rightarrow 32 \rightarrow 24 \rightarrow 16 \rightarrow 24 \rightarrow 32 \rightarrow 48 \rightarrow 64 \rightarrow 74 \quad (9)$$

Deep hidden layers use ReLU activations, and the final reconstruction layer uses Sigmoid activation. Hyperparameters (learning rate = 10^{-3} , batch size = 512, weight decay = 10^{-5} , patience = 8) were held constant to isolate the impact of depth.

D. Threshold Calibration Methodology

Threshold selection directly controls the balance between false positives and false negatives. Rather than arbitrarily setting an absolute threshold, anomaly thresholds τ are calibrated non-parametrically using percentile distributions of reconstruction errors computed over the **unseen normal calibration split** (X_calibration_normal).

Given calibration errors E_{cal} , candidate thresholds are defined as:

$$\tau_p = \text{Percentile}(E_{\text{cal}}, p), \quad p \in \{90.0, 95.0, 97.5, 99.0\} \quad (10)$$

The 95th percentile (p_{95}) serves as the primary operational threshold. This guarantees by construction that the False Positive Rate (FPR) on benign validation traffic remains capped near $1 - 0.95 = 0.05$ (5.0%). During testing, any session with an MSE exceeding $\tau_{p_{95}}$ is flagged as malicious.

V. EXPERIMENTAL EVALUATION & DISCUSSION

A. Training Convergence & Loss Analysis

All three autoencoders were trained for up to 50 epochs with early stopping. Fig. 1 compares training and validation MSE loss curves across the three models.

The basic autoencoder converged smoothly within 25 epochs, stabilizing at a validation MSE of 0.0021. The sparse autoencoder showed similar stability, reaching a validation MSE of 0.0048 while the KL penalty decreased, indicating successful convergence toward the target activation rate $\rho = 0.10$. The deep autoencoder stabilized at a validation MSE of 0.0046 by epoch 38.

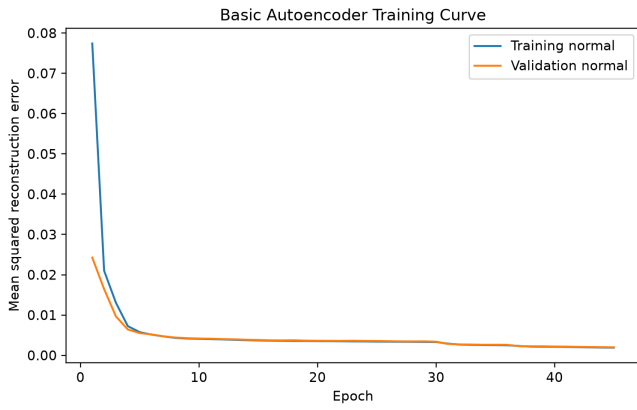


Fig. 1. Training and validation reconstruction loss curves for the basic autoencoder.

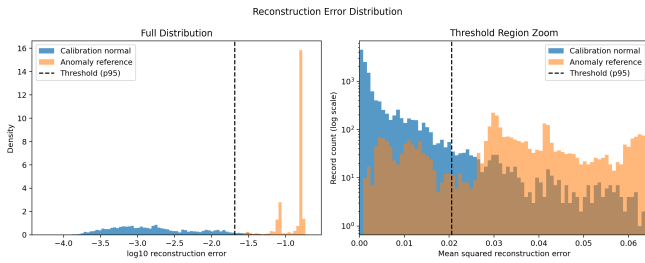


Fig. 2. Reconstruction error distributions for normal calibration records versus anomaly reference records under the sparse autoencoder (log scale).

B. Reconstruction Error Separation

Fig. 2 illustrates the reconstruction error distributions for normal calibration samples versus malicious reference samples produced by the sparse autoencoder.

The distribution demonstrates a clear separation: benign calibration traffic concentrates tightly at low MSE values (mean = 0.005), whereas malicious connection records yield an average MSE of 0.14. This confirms that reconstruction error serves as an effective anomaly indicator.

C. Sensitivity to Percentile Thresholds

Table II details detection metrics across candidate percentile thresholds (p_{90} , p_{95} , $p_{97.5}$, p_{99}) for the sparse autoencoder.

TABLE II
SPARSE AUTOENCODER PERFORMANCE ACROSS PERCENTILE THRESHOLDS

Threshold	Value (τ)	Precision	Recall	F1-Score	Accuracy
p_{90}	0.0134	0.9775	0.9895	0.9834	0.9728
p_{95} (Primary)	0.0205	0.9886	0.9864	0.9875	0.9797
$p_{97.5}$	0.0303	0.9942	0.9774	0.9857	0.9769
p_{99}	0.0447	0.9976	0.9556	0.9762	0.9620

The trade-off follows expected trends: lower percentiles (p_{90}) maximize recall (0.9895) at the cost of higher false alarms. Higher percentiles (p_{99}) achieve high precision (0.9976) but

miss subtle attacks. The primary p_{95} threshold achieves an optimal operational balance (Precision = 0.9886, Recall = 0.9864, F1 = 0.9875).

D. Sparsity Ablation Sweeps (β and ρ)

To evaluate the impact of sparsity regularization, two ablation studies were conducted. Table III displays evaluation metrics when varying sparsity weight β while keeping $\rho = 0.05$ fixed. Setting $\beta = 0$ serves as a non-sparse reference model.

TABLE III
ABLATION SWEEP OVER SPARSITY WEIGHT β ($\rho = 0.05$, p_{95} THRESHOLD)

β Weight	Precision	Recall	F1-Score	R2L Recall	U2R Recall
0 (Non-Sparse)	0.9886	0.9860	0.9873	0.951	1.000
10^{-4}	0.9885	0.9835	0.9860	0.940	0.942
10^{-3}	0.9885	0.9805	0.9845	0.593	0.981
10^{-2}	0.9886	0.9868	0.9877	0.949	0.981
10^{-1}	0.9886	0.9858	0.9872	0.972	0.981

Table IV summarizes the sweep over target sparsity parameter ρ with fixed $\beta = 10^{-3}$.

TABLE IV
ABLATION SWEEP OVER SPARSITY TARGET ρ ($\beta = 10^{-3}$, p_{95} THRESHOLD)

ρ Target	Precision	Recall	F1-Score	R2L Recall	U2R Recall
0.010	0.9886	0.9857	0.9871	0.949	0.981
0.025	0.9886	0.9853	0.9869	0.868	0.981
0.050	0.9885	0.9805	0.9845	0.593	0.981
0.100	0.9886	0.9864	0.9875	0.952	0.981
0.200	0.9885	0.9767	0.9825	0.474	0.981

The ablation results reveal that overall F1 scores remain stable across most settings. However, detection recall for the small, stealthy Remote-to-Local (R2L) attack class is highly sensitive to sparsity tuning. R2L connection records closely resemble legitimate sessions. Setting $\rho = 0.10$ and $\beta = 10^{-3}$ yields the highest R2L recall (0.952) alongside strong overall F1 performance.

E. Cross-Model Comparison Across Attack Categories

Table V contrasts the performance of all three trained models at the primary p_{95} calibration threshold, including per-category recall across DoS, Probe, R2L, and U2R attacks.

All three architectures achieve nearly perfect detection on high volume Denial of Service ($\text{DoS} \geq 99.9\%$) and Surveillance Probe attacks ($\geq 98.1\%$). These attacks feature distinct traffic signatures (e.g., rapid connection rates, unusual port scans) that generate high reconstruction errors.

The primary performance differences appear in the Remote-to-Local (R2L) class:

- The ****Basic Autoencoder**** achieves strong overall performance (F1 = 0.9876, R2L Recall = 0.951).

TABLE V
OVERALL AND PER-CATEGORY PERFORMANCE COMPARISON ACROSS AUTOENCODERS AT PRIMARY $p95$ THRESHOLD

Model Architecture	Precision	Recall	F1 Score	DoS Recall	Probe Recall	R2L Recall	U2R Recall
Basic Autoencoder	0.9886	0.9866	0.9876	0.999	0.982	0.951	1.000
Sparse Autoencoder ($\beta = 10^{-3}, \rho = 0.1$)	0.9886	0.9864	0.9875	0.999	0.982	0.952	0.981
Deep Autoencoder	0.9885	0.9767	0.9825	0.999	0.981	0.510	0.981

- The **Sparse Autoencoder** achieves the highest R2L recall (0.952). Restricting latent unit activations forces the network to learn specialized representations, improving sensitivity to subtle anomalous sessions.
- The **Deep Autoencoder** exhibits a notable performance drop on R2L attacks, with recall falling to 0.510 (51.0%).

VI. METHODOLOGICAL INSIGHTS & DISCUSSION

The experimental findings highlight key trade-offs in autoencoder design for anomaly detection:

A. The Degradation of Unregularized Depth

Increasing network depth without structural regularization (e.g., skip connections or dropout) led to reduced performance on stealthy attack vectors.

Because R2L attacks execute unauthorized commands within standard network protocol sessions, their connection metrics (duration, byte counts, TCP flag sequences) closely align with valid traffic distributions. With 4 dense hidden layers per side, the unregularized Deep Autoencoder possesses high function capacity. As a result, it learns to reconstruct both legitimate traffic and borderline stealthy attack traffic with low reconstruction error, pulling R2L error scores below the $p95$ anomaly threshold ($\tau = 0.0166$).

B. Effectiveness of Bottleneck Sparsity

By contrast, the Sparse Autoencoder enforces latent activation bounds. Forcing most units to remain inactive prevents the network from using excessive parameters to reconstruct borderline patterns. Instead, the model allocates activation capacity strictly to common normal features, causing subtle attack sessions to generate higher reconstruction errors that trigger detection.

VII. CONCLUSIONS & RECOMMENDATIONS FOR FUTURE WORK

This paper presented an empirical evaluation of basic, sparse, and deep autoencoders for unsupervised network intrusion anomaly detection using the KDD Cup 1999 dataset [1]. The results demonstrate that compact, bottlenecked autoencoders effectively learn normal operational baselines, achieving F1 scores up to 0.9876 with low false alarm rates. Enforcing latent sparsity via KL divergence penalties improves detection on low volume, stealthy R2L intrusions. Conversely, increasing depth without structural regularization degrades performance on subtle attack vectors.

Future work will focus on:

- 1) Incorporating residual skip connections and batch normalization in deep autoencoder architectures to prevent gradient degradation.
- 2) Evaluating Variational Autoencoders (VAEs) to establish probabilistic anomaly scoring boundaries.
- 3) Incorporating Recurrent (LSTM/GRU) or Transformer layers to model temporal dynamics across sequential connection bursts.
- 4) Validating the pipeline on modern benchmark datasets, such as CIC-IDS2017 [11] and UNSW-NB15 [6].

REFERENCES

- [1] S. Stolfo, W. Fan, W. Lee, A. Prodromidis, and P. Chan, "Cost-based modeling for fraud and intrusion detection," *DARPA Information Survival Conference and Exposition*, vol. 1, pp. 130–144, 2000.
- [2] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," *arXiv preprint arXiv:1802.07389*, 2018.
- [3] M. Sakurada and T. Yairi, "Anomaly detection using autoencoders with nonlinear dimensionality reduction," *Proceedings of the MLSDA Workshop*, pp. 4–11, 2014.
- [4] A. Ng, "Sparse autoencoder," *CS294A Lecture Notes*, Stanford University, pp. 1–19, 2011.
- [5] K. Alrawashdeh and C. Purdy, "Toward online anomaly detection in network traffic using deep learning," *IEEE International Conference on Machine Learning and Applications (ICMLA)*, pp. 194–200, 2016.
- [6] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," *Military Communications and Information Systems Conference (MilCIS)*, Canberra, Australia, pp. 1–6, 2015.
- [7] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [8] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, pp. 8024–8035, 2019.
- [9] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.
- [10] M. Abadi *et al.*, "TensorFlow: Large-scale machine learning on heterogeneous systems," *Software available from tensorflow.org*, 2015.
- [11] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, Funchal, Portugal, 2018, pp. 108–116.